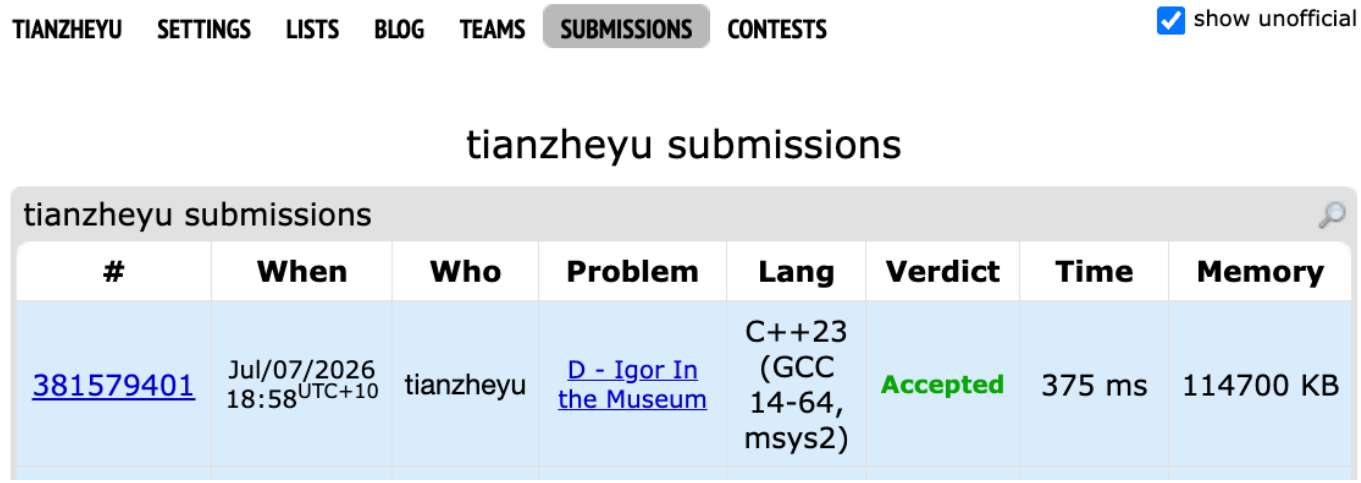
Problem Set 5

B. Igor In the Museum



https://codeforces.com/submissions/tianzheyu#

Process

To avoid recalculating paths for every query, we can precompute the answers. We can treat the grid as a graph and use DFS to find all connected components of empty cells (`.`). During the traversal, we assign a unique component ID to each visited empty cell and count the total number of adjacent walls. This allows each query to be answered in $O(1)$ time by simply looking up the precalculated wall count for the given starting cell's component.

Challenges and Overcoming

When implementing the DFS, I ran into a logical bug regarding how to count the pictures. Initially, whenever my DFS reached an impassable wall cell (`*`), I incremented the picture count and marked that specific `*` cell as `visited` in my matrix. I found the bug when my output was consistently lower than the expected answer on the sample tests. I realized that a single `*` cell can share multiple boundaries with the empty space (e.g., a wall block jutting out shares three sides), and the problem explicitly states that *every* shared border has a picture. By marking the `*` cell as `visited`, I was counting the physical grid block only once instead of counting the edges. I fixed this by returning immediately upon hitting a `*` without updating the `visited` array. Next time I work on grid traversal, I will explicitly define whether the target metric is node-based (blocks) or edge-based (borders) before writing my state-tracking logic.